ONE CLAW 龙虾可视化控制面板

技术架构分析报告

项目名称	框架版本
openclaw-bot-review	Next.js 16 / React 19

报告日期	分析模块数
2026-03-29	100+ 文件

目 录

一 整体技术栈

二 总体架构图

三 目录结构与职责

四 前端页面路由与功能模块

五 后端 API 路由分层设计

六 Lib 层工具库职责

七 数据流与状态管理

八 与外部系统的集成点

九 像素办公室独立架构

十 部署架构

十一 关键设计模式与架构决策

十二 模块复杂度与代码规模

十三 架构总结

一 整体技术栈

前端框架	全栈框架	类型系统
React 19	Next.js 16	TypeScript 5

样式	关系型数据库	嵌入式数据库
Tailwind CSS 4	MySQL 8.0	SQLite3

层次	技术	版本	用途
前端框架	Next.js App Router	16.0.0	SSR/CSR 全栈框架，内置 API Routes
UI 库	React	19.0.0	组件化 UI，Hooks 状态管理
类型系统	TypeScript	5.0.0	静态类型检查，提升代码可维护性
样式系统	Tailwind CSS + PostCSS	4.0.0	原子化 CSS，5 套主题系统
运行时	Node.js Alpine	22.x	服务端执行环境
关系型数据库	MySQL	8.0+	长期归档、聚合统计、Agent/Channel 存储
嵌入式数据库	SQLite3 (better-sqlite3)	12.8.0	Agent 本地会话记忆、高频写入
图像处理	Sharp	0.34.5	生成像素办公室资源
部署容器	Docker	—	多阶段构建，standalone 最小镜像
桌面应用	Electron	—	可选打包为 .exe/.dmg

二 总体架构图

```
┌──────────────────────────────────────────────────────────────┐
│                    浏览器（React 客户端）                      │
│  ThemeProvider → I18nProvider → Sidebar + AlertMonitor       │
│                                                              │
│  ┌──────────┐ ┌──────────┐ ┌─────────┐ ┌─────────┐          │
│  │Dashboard │ │PixelOffice│ │ Stats   │ │ Alerts  │ ...      │
│  └──────────┘ └──────────┘ └─────────┘ └─────────┘          │
└──────────────── fetch / WebSocket ──────────────────────────┘
                              │
┌──────────────────────────────────────────────────────────────┐
│             Next.js API Routes  (app/api/)                   │
│  /config  /stats  /alerts  /pixel-office  /chat  /setup      │
│  /memory  /sessions  /openclaw  /storage  /test-*            │
│                   ↑ localhost 守卫 (api-local-guard)         │
│                                                              │
│             lib-layer  (业务逻辑层)                           │
│  openclaw-cli  model-probe  db-sync  alert-incidents         │
│  gateway-url   config-cache  openclaw-paths  i18n  theme     │
└──────────────┬──────────────┬──────────────┬────────────────┘
               │              │              │
    ┌──────────┴──────────┐ ┌─┴──────────┐ ┌─┴──────────────────┐
    │  文件系统           │ │  MySQL     │ │  OpenClaw Gateway  │
    │ ~/.openclaw         │ │ oc_agents  │ │  ws://127.0.0.1:18789 │
    │ *.json              │ │ oc_sessions│ │  CLI: openclaw ...  │
    │ *.jsonl             │ │ oc_channels│ │  HTTP /v1/chat/...  │
    │ *.sqlite            │ └────────────┘ └────────────────────┘
    └─────────────────────┘
```

三 目录结构与职责

app/ — 前端页面与 API 路由

🗂️ **layout.tsx**	根布局 — 挂载 Sidebar、AlertMonitor、GlobalBugsOverlay
🎨 **providers.tsx**	ThemeProvider + I18nProvider，全局 Context 注入
🧭 **sidebar.tsx**	可折叠侧边栏，包含 PixelSVG 图标系统、Logo 拖拽交互
🔔 **alert-monitor.tsx**	全局后台静默轮询告警，非 /setup 路径均运行
🦎 **global-bugs-overlay.tsx**	像素虫子全局覆盖层（游戏化装饰）

app/api/ — 后端 API（Next.js API Routes）

```
app/api/
├── config/            配置层：获取/修改 Agent、Provider、Model 配置
├── openclaw/          OpenClaw 层：status、gateway-status、doctor、restart、channels
├── stats/             统计层：单Agent统计、全局汇总、模型维度
├── storage/           存储层：MySQL 指标查询、OpenClaw→MySQL 同步
├── memory/            记忆层：SQLite实时查询、Markdown 归档读写
├── pixel-office/      游戏层：房间布局、Agent贡献、BGM轨道
├── alerts/            告警层：配置读写、触发检查
├── chat/              聊天层：completions（流式）、WebSocket升级
├── sessions/          会话层：单Agent会话列表
├── agent-activity/    活动层：Agent 任务/Cron 追踪
└── test-*/            测试层：模型/平台/会话/DM 连接测试
```

lib/ — 业务逻辑工具库

```
lib/
├── openclaw-cli.ts           OpenClaw CLI 命令执行（spawn + stream）
├── openclaw-paths.ts         路径常量（OPENCLAW_HOME, AGENTS_DIR 等）
├── openclaw-home-detect.ts   自动探测 OpenClaw 主目录（多路径候选）
├── openclaw-skills.ts        技能列表读取与解析
├── gateway-url.ts            Gateway WebSocket URL 构建
├── i18n.tsx                  国际化（6语言：zh/zh-TW/en/ms/id/th）
├── theme.tsx                 主题系统（5套 CSS 变量主题）
├── db-sync.ts                OpenClaw JSON → MySQL 5张表同步
├── db-metrics.ts             MySQL 统计指标查询
├── alert-incidents.ts        告警事件状态机（active/ack/snoozed/recovered）
├── model-probe*.ts           模型连接探针（OpenAI兼容 + 自定义HTTP）
├── agent-session-activity.ts 会话活跃性分析（多源合并）
├── config-cache.ts           配置内存缓存（30s TTL）
├── api-local-guard.ts        localhost 访问守卫（白名单 IP）
```

四 前端页面路由与功能模块

路由	功能描述	主要数据源
/oneone-dashboard	主仪表盘：网关状态、版本、通道数、存储指标、诊断工具	/api/openclaw/status, /api/stats-all
/ (expert-squad)	专家战队：Agent 卡片、模型测试、平台/会话测试、任务追踪	/api/config, /api/agent-status
/pixel-office	🎮 游戏化可视化：像素办公室、Agent化身漫步、家具编辑器	/api/pixel-office/*
/models	模型管理：Provider 配置、性能指标、模型探针测试、添加自定义模型	/api/config, /api/stats-models
/sessions	会话浏览：频道分布、Agent 分布、搜索/过滤	/api/sessions/[agentId]
/channels	通道管理：飞书/钉钉/Discord/Telegram 通道可视化配置	/api/openclaw/channels
/memory	记忆管理：SQLite 实时数据 + Markdown 归档文件编辑	/api/memory/*
/expert-files	专家文件管理：浏览、搜索、编辑 Agent 文件	/api/expert-files
/stats	消息统计：日/周/月聚合、Token 消耗趋势、响应时间分析	/api/stats/[agentId]
/alerts	告警中心：规则配置、事件闭环（确认/静默/恢复）	/api/alerts/*
/skills	技能管理：内置/扩展/自定义技能查看与管理	/api/skills/*
/setup	部署向导：预检查、一键安装、模型/通道初始化配置	/api/setup/*
/chat	Web Chat：实时对话、WebSocket 连接、会话管理	/api/chat/*

📌 路由说明

- 根路径 `/` 通过 next.config.mjs 重定向到 `/oneone-dashboard`
- 根页面 `app/page.tsx` 实际渲染的是专家战队（Expert Squad）内容
- 侧边栏（sidebar.tsx）包含 4 个导航分组：总览 / OneOne管理 / 监控 / 配置
- 所有页面均为 Client Component，采用 useEffect + fetch 获取数据

五 后端 API 路由分层设计

分层	路由	方法	说明
配置层	/api/config	GET	整合 Agent/Provider/Model，30s 缓存
	/api/config/add-provider-model	POST	添加模型 Provider
	/api/config/agent-model	PATCH	修改 Agent 绑定模型
状态层	/api/openclaw/status	GET	网关版本、通道数、模型数
	/api/gateway-health	GET	网关健康检查
	/api/agent-status	GET	Agent 运行状态（working/idle/offline）
命令层（流式）	/api/openclaw/doctor	POST	诊断修复，Server-Sent Events 流
	/api/openclaw/restart	POST	重启 Gateway
	/api/setup/oneclick-install	POST	一键安装，流式进度输出
统计层	/api/stats/[agentId]	GET	单 Agent 日/周/月统计
	/api/stats-all	GET	全局统计汇总
	/api/stats-models	GET	模型维度统计
	/api/activity-heatmap	GET	Agent 活跃热力图
聊天层	/api/chat/completions	POST	聊天接口（支持流式 SSE）
	/api/chat/connect	GET	WebSocket 协议升级
存储层	/api/storage/metrics	GET	MySQL 指标查询
	/api/storage/sync	POST	手动触发 OpenClaw → MySQL 同步
告警层	/api/alerts	GET/PUT	告警配置读写
	/api/alerts/check	POST	触发告警检查，发送飞书通知

🔒 安全设计

- 所有敏感 API 使用 `enforceLocalRequest()` 限制仅 localhost 可访问
- 可通过 `SETUP_ALLOW_REMOTE=1` 环境变量解除限制（生产环境慎用）

六 Lib 层工具库职责

6.1 OpenClaw 集成模块

文件	职责
openclaw-cli.ts	执行 `openclaw` CLI（spawn + stream，支持 .mjs 直调避 Windows cmd 问题）
openclaw-paths.ts	路径常量：OPENCLAW_HOME、OPENCLAW_CONFIG_PATH、OPENCLAW_AGENTS_DIR 等
openclaw-home-detect.ts	自动探测 OpenClaw 主目录（多路径候选 + 环境变量优先）
openclaw-skills.ts	读取并解析技能列表（内置/扩展/自定义分类）
gateway-url.ts	构建 Gateway WebSocket URL（支持 host/port 覆盖）
im-channel-config.ts	IM 通道配置解析（飞书/钉钉/Discord/Telegram 等）

6.2 数据库模块

文件	职责
mysql.ts	MySQL 连接池（上限 10 连接，60s 空闲释放，环境变量配置）
db-sync.ts	OpenClaw JSON → MySQL 双向同步，写入 5 张表（Config/Agents/Channels/Sessions/SyncRuns）
db-metrics.ts	查询 MySQL：Agent计数、Session分布、通道分布、同步历史记录

6.3 监控与告警

文件	职责
alert-incidents.ts	告警事件状态机：critical/warning/info 级别，active/acknowledged/snoozed/recovered 状态流转
agent-session-activity.ts	会话活跃性多源合并（sessions.json + 目录 mtime + Gateway RPC）
gateway-agent-activity-hint.ts	从 Gateway 获取 Agent 活动快照

6.4 通用工具

文件	职责
config-cache.ts	配置内存缓存，30s TTL，避免频繁磁盘 IO
api-local-guard.ts	localhost 访问守卫，检查 host header
model-probe*.ts	模型连接探针，支持 OpenAI 兼容接口和自定义 HTTP
json.ts	JSON 安全解析/序列化（容错处理）
gateway-reachability.ts	Gateway 可达性检查（带超时）

七 数据流与状态管理

7.1 整体数据流向

```
用户操作（点击/搜索/设置）
      ↓
React 组件（useState + useEffect）
      ↓
fetch 调用 → Next.js API Route
      ↓
lib 工具函数（业务逻辑处理）
      ↓
多源数据读写：
├── 文件系统：~/.openclaw/openclaw.json、agents/*/sessions/
├── MySQL：oc_agents、oc_sessions、oc_channels 等
├── SQLite：agents/*/memory/*.sqlite（同步读）
└── OpenClaw CLI：spawn 执行命令
      ↓
JSON 响应 → React state 更新 → UI 重渲染
```

7.2 状态管理策略

策略	适用场景	实现
本地 useState	页面级状态（加载中/错误/搜索/分页）	React Hooks
全局 Context	主题、语言（跨页面共享）	ThemeProvider + I18nProvider
localStorage	用户偏好持久化（主题/语言/测试结果）	浏览器存储
服务端内存缓存	配置数据（30s TTL）	config-cache.ts
定时轮询	Agent状态(30s)、告警检查(可配置间隔)	setInterval
WebSocket 实时	Web Chat 实时对话	/api/chat/connect

7.3 缓存层次

```
浏览器层  localStorage（主题、语言、测试结果，永久）
    ↓ miss
API 层    内存缓存（配置数据，30s TTL）
    ↓ miss
磁盘层    fs.readFileSync（~/.openclaw/*.json）
    ↓ 异步写入
数据库层  MySQL 连接池（oc_agents/oc_sessions 等）
```

八 与外部系统的集成点

系统	集成方式	用途
OpenClaw Gateway	HTTP + WebSocket (127.0.0.1:18789)	实时聊天、Agent活动快照、网关状态
OpenClaw CLI	spawn('openclaw ...')	诊断修复、网关重启、配置查询
飞书 / Lark	HTTP API (open.feishu.cn / open.larksuite.com)	告警通知推送（DM 发送）
MySQL	连接池 (mysql2)	长期归档存储、聚合统计、Agent快照
SQLite	better-sqlite3 同步读	Agent 本地会话记忆、实时数据查看
AI Providers	自定义 HTTP 探针（OpenAI 兼容）	模型可用性测试、延迟检测

8.1 多 IM 通道支持

飞书 Feishu　钉钉 DingTalk　Discord　Telegram　WhatsApp　QQBot

8.2 多 AI Provider 支持

Anthropic Claude　OpenAI GPT　Google Gemini　月之暗面 Kimi　Ollama（本地）　自定义 HTTP 端点

8.3 飞书告警通知流程

```
AlertMonitor（后台轮询）
      ↓
POST /api/alerts/check
      ↓
checkModelAlerts + checkBotResponseAlerts + checkCronAlerts
      ↓
reconcileIncidents（状态机：active→acknowledged→snoozed→recovered）
      ↓
shouldSendNotification（防重发：1分钟内不重复）
      ↓
sendAlertViaFeishu:
  1. 获取 tenant_access_token（app_id + app_secret）
  2. 查找最近 DM 用户（open_id）
  3. POST /open-apis/im/v1/messages（user_id → open_id fallback）
```

九 像素办公室独立架构

像素办公室是项目中最复杂的模块（40+ 文件），采用独立于普通 Web 页面的 Canvas 渲染架构，运行 60fps 游戏循环。

9.1 核心引擎架构

```
Canvas 60fps 渲染循环（requestAnimationFrame）
              ↓
    gameLoop.ts ──→ officeState.ts（全局状态）
              ↓
    renderer.ts（Canvas 像素图形渲染）
              ↓
    ┌──────────────────────────────────┐
    │ characters.ts  (Agent 化身角色)      │
    │ wanderProfiles.ts (漫步行为 AI)      │
    │ bugSystem.ts (虚拟 Bug NPC)          │
    │ matrixEffect.ts (Matrix 粒子特效)    │
    └──────────────────────────────────┘
              ↓
    agentBridge.ts ←── /api/pixel-office/*
    （同步真实 Agent 活跃状态到画面）
```

9.2 子系统总览

子系统	文件	功能
游戏引擎	engine/gameLoop.ts, renderer.ts	帧循环、Canvas 2D 像素渲染
角色系统	engine/characters.ts, wanderProfiles.ts	Agent 化身、漫步 AI 行为树
精灵系统	sprites/spriteCache.ts, pixelAssetPack.ts	精灵缓存、像素资源包加载
布局系统	layout/layoutFactory.ts, furnitureCatalog.ts	5 种主题房间、家具目录
编辑器	editor/editorState.ts, editorActions.ts	家具放置/移除/旋转交互
Bug 系统	bugs/bugSystem.ts, pheromoneField.ts	虚拟 Bug NPC、信息素场寻路
音频系统	audio/notificationSound.ts, proceduralBgm.ts	通知音效、程序化 BGM
Agent 桥接	agentBridge.ts	真实 Agent 活动数据 → 游戏状态同步

部署架构

10.1 Docker 多阶段构建

```
FROM node:22-alpine AS builder
    npm ci
    npm run build
    → .next/standalone/ (最小化输出)

FROM node:22-alpine AS runner
    COPY .next/standalone ./
    COPY .next/static ./.next/static
    COPY public ./public
    EXPOSE 3000
    CMD ["node", "server.js"]

镜像大小：约 150-200MB（standalone 模式，无 node_modules）
```

10.2 部署方式对比

部署方式	适用场景	端口	关键命令
npm run dev	本地开发	3003	node scripts/check-dev-port.mjs && next dev
Docker	云端服务器 / K8s	3000	docker build + docker run
Next.js standalone	EXE/DMG 打包基础	3003	npm run packaging:prepare-standalone
Electron	桌面应用	内嵌	packaging/electron/ 目录

10.3 关键环境变量

```
OPENCLAW_HOME=/home/user/.openclaw       # OpenClaw 配置目录（必须）
OPENCLAW_CLI=openclaw                    # CLI 可执行路径
MYSQL_HOST=127.0.0.1                     # MySQL 连接（可选，有默认值）
MYSQL_PORT=3306
MYSQL_USER=root
MYSQL_PASSWORD=root
MYSQL_DATABASE=openclaw_visualization
SETUP_ALLOW_REMOTE=0                     # 是否允许远程访问 setup API
LOBBY_SETUP_SECRET=your-secret           # 部署向导密钥保护
```

十一 关键设计模式与架构决策

11.1 设计模式应用

模式	应用场景	实现文件
Provider Pattern	全局状态（主题/语言）注入	providers.tsx, i18n.tsx, theme.tsx
Factory Pattern	生成不同主题的房间布局	pixel-office/layout/layoutFactory.ts
Observer Pattern	告警监听、Agent 活动追踪	alert-monitor.tsx, agent-activity.ts
State Machine	告警事件生命周期管理	alert-incidents.ts
Cache-Aside	配置数据 30s 缓存	config-cache.ts
Streaming Pattern	CLI 命令流式进度输出	/api/openclaw/doctor, /api/setup/*
Fallback Pattern	数据获取失败保留旧快照	app/page.tsx（cachedHomeData）
Guard Pattern	localhost 访问控制	api-local-guard.ts

11.2 关键架构决策

为何选择 Next.js 全栈而不是前后端分离?

- API Routes 内置，无需单独后端服务，简化部署为单进程
- standalone 输出模式支持轻量 Docker 镜像（~150MB）
- 文件路由约定简洁，页面与 API 共享类型定义

为何 MySQL + SQLite 混合存储?

- MySQL：长期归档、聚合查询、跨 Agent 汇总报表
- SQLite：Agent 本地高频写入、零配置、无需网络连接
- 两者互补，文件系统是"真相来源"，MySQL 是"分析层"

像素办公室为何与普通页面完全解耦?

- Canvas 渲染管道与 React DOM 渲染互不干扰
- 60fps 游戏循环需要独立 requestAnimationFrame 管理
- 复杂状态（家具/角色/碰撞）不适合 React state 管理

十二 模块复杂度与代码规模

模块	文件数（估）	核心职责	复杂度
pixel-office/	40+	Canvas 游戏引擎、物理、行为树	🔴 高
app/api/	40+	数据接口、CLI 集成、聚合逻辑	🟡 中高
lib/（非像素）	20+	配置解析、DB 同步、告警逻辑	🟡 中
app/pages/	12+	UI 表单、图表、列表展示	🟢 低-中
i18n + theme	5	多语言翻译表、CSS 变量主题	🟢 低
scripts/	6	构建辅助脚本	🟢 低

12.1 国际化覆盖

CN 简体中文 zh　TW 繁体中文 zh-TW　US 英文 en　MY 马来文 ms　ID 印尼文 id　TH 泰文 th

12.2 主题系统

🌊 深海暗夜　⚡ 极简浅色　💙 蓝色科技风　🔥 橙色暖调　🌿 森林绿境

十三 架构总结

ONE CLAW 龙虾可视化控制面板是一个以 OpenClaw 为核心、以 Next.js 为全栈框架的 AI Agent 运维管理面板。项目最大的特色是将严肃的运维功能（监控/告警/统计/配置）与游戏化可视化（像素办公室）深度融合。

核心亮点

方面	设计	价值
全栈合一	Next.js API Routes 无独立后端	单进程部署，运维简单
双数据库	MySQL 归档 + SQLite 实时	读写特性互补，零依赖本地运行
流式输出	SSE 给 CLI 命令	长时操作实时进度反馈
游戏化	Canvas 引擎与 DOM 解耦	60fps 高性能，趣味化运维
多语言/主题	6语言 + 5主题	全球化支持，高适配性
安全隔离	localhost 守卫	防止未授权的远程访问
多部署方式	Docker / Electron / standalone	云端 + 桌面双模支持

✅ 架构优势

- 模块化清晰：页面/API/工具库三层分离，职责明确
- 扩展性强：插件式模型/通道/技能支持，新增 Provider 无需改核心
- 容错完善：数据获取失败降级、静默重试、多源活跃度合并
- 国际化完整：6语言全量覆盖，含东南亚小语种
- 部署灵活：从开发机 npm dev 到 Docker K8s 到桌面 Electron